

Polterheist - Team34

Polterheist - Team34 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A mobile perspective-switching puzzle game (in the vein of *Toodee and Topdee*). You play a kleptomaniac ghost robbing the underworld: the main body is controlled in a side-view (tap to move toward a point), while the "spirit" sees the world in a top-down $\frac{3}{4}$ view, flies undetected, and possesses blocks to move them around. Get the body to the target without detection to complete the heist. Named technical features: side-view ↔ top-down isometric perspective switching (player controls the viewing angle); a data-driven engine tailored to designer needs; a server-side solution for leaderboards and social functions; procedural map generation from preset sections. Custom tools: a game editor, runtime debug tools, and a custom map/room editor for the procedural generation. Design features: the perspective-switching mechanic (reshape the stage from the top-down view); ≥ 5 possessable block types with unique stage interactions; ≥ 5 designed stages using the perspective/gravity switch; ≥ 3 enemies with distinct behaviours. Target 16 ms framerate.

References:

- [Toodee and Topdee on Steam](#) (the stated inspiration — a 2D-platformer × top-down-puzzler hybrid; switch perspectives, and the world bends to the current genre's rules)
- [FEZ — Wikipedia](#) (perspective-shifting 2D/3D puzzle-platformer built on rotating between orthogonal views)
- [Monaco: What's Yours Is Mine on Steam](#) (top-down stealth heist — case the joint and steal without detection, with leaderboards and a level editor)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

Core Tech Requirement	Polterheist
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✗
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — tap-to-move and tap-to-possess are touch-first and the $\frac{3}{4}$ top-down/side switch suits a phone screen (device: Pixel 9a); the Native PC build is the development/tools vehicle. **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: Dual renderer — a side-view of the body and an isometric $\frac{3}{4}$ top-down view of the spirit and the level — with the perspective-switch transition, possessed-block highlighting, detection/stealth cues and stage HUD (OpenGL ES). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten,

SDL2/3, OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: Engine core (fixed + variable timestep loop targeting 16 ms, entity system, per-perspective physics/collision & gravity, block-possession system, enemy AI & detection, procedural room assembler) + a leaderboard/social service. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — leaderboard/social seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it advances the body (side-view movement to a tapped point), the spirit (free top-down flight + block possession/movement), per-perspective physics and gravity, enemy behaviour and detection, and the perspective-switch state, holding a 16 ms frame budget with mixed fixed/variable timesteps.
- *Data-driven:* stages, the ≥ 5 block types and their interactions, the ≥ 3 enemies and their behaviours, preset rooms for procedural generation and the perspective/gravity rules are all hot-reloadable JSON authored via the editors — a data-driven engine tailored to the designers.
- *2D:* sprite/tile rendering of both the side-view and the isometric $\frac{3}{4}$ top-down view, the switch transition, possession and detection VFX (custom-rendered). *3D:* not needed (the $\frac{3}{4}$ view is 2D/iso sprites). *Networking:* async leaderboards / social via Firebase; no real-time netcode.
- *Custom tool:* a game editor + a custom **map/room editor** (author preset rooms for procedural assembly, place blocks, spawns, targets and detection zones) with live preview, plus runtime debug tools.
- *External DB:* leaderboards and social functions. *Auth* »: accounts for leaderboards/social. *Analytics* »: optional — where players are detected/stuck, for stage balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window with tap-to-move / tap-to-possess input and a mixed fixed+variable timestep loop targeting 16 ms that advances the body, spirit, physics and the perspective-switch state; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G5 physics and M1G8 enemy detection run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component registry in C++ with stages, the block types and interactions, enemies, preset rooms and the perspective/gravity rules defined in JSON (nlohmann/json) with hot-reload — the data-driven engine the designers author against.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M2G2 (the editor edits this data), M2G14 (procgen consumes rooms), M1G8 (enemy/data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui runtime debug tools (perspective/gravity state, possession, detection cones), Tracy profiling to hold the 16 ms budget, Catch2 stress tests (many blocks + enemies + a perspective switch under load), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).

- **Feeds into:** M2G3 (CI hardening) and the frame budget M2G1 validates on-device; the runtime debug tools are a named deliverable.
- **G4 2D Graphics** — the dual side-view + isometric $\frac{3}{4}$ top-down renderer and the perspective-switch transition are the whole read of the game.

Tech & dependencies

- **Tech: Web build:** Sprite/tile renderer with two projections (side + iso $\frac{3}{4}$ top-down), a camera, batching, and the switch transition; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType HUD text. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (advanced switch/possession VFX); every visible feature of the demo.
- **G5 Physics & Collision** — per-perspective movement, gravity switching and block possession/pushing are the mechanical core.

Tech & dependencies

- **Tech:** C++ + GLM math, a custom integrator handling side-view body movement and top-down spirit flight, gravity switching, and collision/resolution when possessed blocks move and reshape the stage; broadphase (grid/quadtrees).
- **Prerequisites:** G1 (loop & tick), G2 (block data).
- **Feeds into:** The puzzle interactions; M2G14 (procgen rooms must be solvable under these rules).
- **G8 Enemy AI & Pathfinding** — the ≥ 3 enemy types with distinct behaviours and the detection that makes the heist a stealth puzzle.

Tech & dependencies

- **Tech:** Finite-state machines and A*/steering for the enemies, detection cones/awareness against the body and spirit, tuned per enemy type from data (custom C++).
- **Prerequisites:** G1 (tick), G2 (enemy data), G5 (movement/collision).
- **Feeds into:** M2G10-class advanced behaviour; the "without detection" stealth loop.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling to hold 16 ms through a perspective switch, ASan on MSVC, memory pools for blocks/enemies, and Emscripten WASM validation for the mobile-web target plus on-device Pixel 9a checks; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).
 - **Feeds into:** The web/mobile deliverable and a stable frame budget across both perspectives.
- **G2 ★ Content Editor — Core** — the game editor + the custom map/room editor: author stages, place blocks/spawns/targets/detection zones and the preset rooms for procedural generation.

Tech & dependencies

- **Tech:** ImGui editor authoring stages, block placement, enemy spawns, targets and preset rooms as JSON, with round-trip serialization, live preview in both perspectives and an undo/redo command stack.
 - **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (dual-perspective live preview).
 - **Prerequisites:** G2 (stage/block/room data), M1G4 (rendering for preview).
 - **Feeds into:** M2G6 (advanced room-authoring), M2G14 (procgen consumes the rooms).
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the native build plus the Emscripten web build artifact via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every Polterheist goal.

- **G6 Content Editor — Advanced Tool Features** — the room editor matured: drag-drop room authoring with a live preview that validates a room is solvable under both perspectives.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets, drag-drop room/block editing, live preview binding that runs the sim inside the tool, and deep undo/redo.
- **Prerequisites:** M2G2 (editor core), M1G5 (sim used for solvability preview).
- **Feeds into:** The preset-room library that M2G14 assembles; the content-creation showcase.

- **G14 Procedural Generation II** — assemble the preset rooms into varied, solvable stages with pacing.

Tech & dependencies

- **Tech:** Constraint-based assembly of preset rooms (connectivity, difficulty pacing), seeded RNG for reproducibility, and automated validation that generated stages are completable under the perspective/gravity rules.
- **Prerequisites:** M2G2/M2G6 (the room library), M1G5 (solvability rules).
- **Feeds into:** Replayable content variety at the showcase.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — the server-side leaderboards and social functions.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore leaderboards (best time / least detection), optional social functions, and Analytics events, via the REST API from C++.
- **Prerequisites:** G2 (run/score data model).
- **Feeds into:** Competitive leaderboards and the social layer; stage balancing telemetry.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — every block, room and enemy needs art in **two** projections (a side view and an isometric $\frac{3}{4}$ top-down view), so the environment/character art roughly doubles and earns 2 artists; a solo designer owns the perspective/gravity/possession puzzle design across the ≥ 5 stages and the block-interaction matrix (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If stage/puzzle volume outgrows one designer, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* perspective switch + gravity + ≥ 5 blocks + ≥ 5 stages + ≥ 3 enemies + procgen + editor + server is a large surface — treat the perspective-switch + possession core with 2–3 hand-built stages and the editor as the M1/M2 floor, with procedural assembly and the server layer as the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the switch → possess → reshape → sneak-to-target heist loop; benchmark e.g. Toodee and Topdee for perspective-swap clarity)
- M1D2 Core Gameplay Loop Prototype (greybox proving the side/top switch + block possession makes a satisfying puzzle) · prototype in Unity
- M1D6 Level & Progression Design (the ≥ 5 stages, the block-interaction matrix, and the teach-test-twist curve for the switch/gravity mechanic)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (stage difficulty, detection tuning and block behaviour via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the two perspectives, the switch affordance, possession and detection cues.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — standard screens + the switch/possession controls navigable unaided on the mobile target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD and touch controls on the Pixel 9a.

- **Art M1:** 2 artists justified (every asset in two projections + characters + enemies)

- M1A1 ★ (mockups: main menu + a stage shown in both perspectives)
- M1A2 Characters (the ghost body, the spirit, + the 3 enemy types, silhouette-first)
- M1A3 Environment (the underworld tileset in BOTH side and iso $\frac{3}{4}$ top-down projections)
- M1A4 UI Art Concepts & Typography (switch affordance, possession/detection HUD)
- M1A5 Animation Foundations (body move, spirit float, possession, enemy idle/alert)
- M1A6 Early Asset Production

- **Art M2:**

- M2A1 ★
- M2A2 Extensive Animation (perspective-switch transition, possession, enemy behaviours)
- M2A3 Environment Alive (both-projection ambient motion, block/room variety)
- M2A4 UI Art — States & Feedback (detected, possessing, switch-locked, target reached)
- M2A5 VFX — Effects & Transitions (the switch morph, possession glow, detection stingers)
- M2A6 Cutscenes & Storyboards (heist intro / outro)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (spooky underworld BGM, possession, switch, detection stings), suitable and correctly triggered.

- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (atmospheric BGM + possession / switch / detection / heist-complete SFX, ≥6 types).
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a cue for the perspective switch).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the map/room editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*